

Применение Awk для разбора текстовых файлов с данными

Доклад

Жукова С. В. НПИбд-01-24



24 марта 2024

- Жукова София Викторовна
- студентка
- направления прикладной информатика
- Российский университет дружбы народов
- 1032240966@pfur.ru
- <https://svzhukova.github.io/ru/>



Вводная часть

Содержание исследования

- Актуальность
- Объекты и предмет исследования
- Цели и задачи
- Основная часть
- Результаты и выводы



- **Актуальность** - Умение эффективно анализировать и обрабатывать данные - ключевой навык для специалистов в различных областях
- **Объекты и предмет исследования** - Текстовые файлы с данными и методы работы с ними
- **Цели и задачи**
- **Основная часть**
- **Результаты и выводы**



Примеры простых команд Awk:

```
awk '{ print }' file
```

```
awk '{ print $1 }' file
```

```
awk 'NR == 1 { print }' file
```

- `awk 'pattern { action }' filename`
- **pattern:** условие, при выполнении которого будет срабатывать действие.
- **action:** команда или набор команд, исполняемых для строки, удовлетворяющей условию.



По умолчанию awk использует пробелы и табуляцию как разделители полей. Каждый текстовый файл разбивается на поля, которые можно обратиться с помощью переменных:

- \$1, 2, ..., `
- n' (первое, второе и т.д. поле).
- NF — общее количество полей в текущей строке.
- FS — переменная для задания разделителя.



Условия - Операторы `if`, `else if`, `else` позволяют выполнять разные действия.

Циклы - `for`, `while`, `do-while` обеспечивают повторение операций.

Сравнения - Операторы `==`, `!=`, `>`, `<`, `>=`, `<=` сравнивают значения.

- `awk '{ if (`
- `$1 > 100) print $`
- `1 }' data.txt`



Ассоциативные Массивы в Awk

Ассоциативные массивы позволяют хранить данные в формате ключ-значение. Это особенно полезно для подсчета уникальных значений.

Пример:

```
{
    count[
$1]++
}
END {
    for (word in count) {
        print word, count[word]
    }
}
```



Встроенные Функции Awk

Awk предоставляет множество встроенных функций для работы со строками, математикой и временем. Некоторые из них:

- **length(string)**: возвращает длину строки.
- **substr(string, start, length)**: извлекает подстроку.
- **split(string, array, sep)**: разделяет строку на массив с указанным разделителем.



Подсчет строк и полей в текстовом файле:

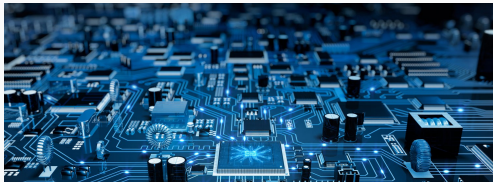
- `awk 'END { print "Строки:", NR, "Поля:", NF }' file.txt`

Фильтрация данных:

- `awk '$`
- `3 > 50' data.txt`

Суммирование значений:

- `awk '{ sum +=`
- `$2 } END { print sum }' file.txt`



Закключение

Использование awk позволяет существенно упростить процессы работы с текстовыми данными и повысить эффективность анализа. Освоение этого инструмента будет полезно как для начинающих, так и для опытных специалистов в области обработки данных.

